

Yesterday 7:15 PM

developpe et inspire toi de cette discussion riche en sens et concept ingestion de document et de meta cognition :

, pendant le trajet ,euh bah du coup je pense à toi et euh Et je me dis ok, il faut que tu lui parles, faut que je lui demande un truc euh. uh, J'ai ça sur le sur le bout de l'esprit donc faut que faut que je te développe

D'accord, vas-y, je t'écoute.

uh, Tu sais, ce qu'on a parlé ce matin par rapport à notre modèle. Et du coup, uh au niveau de réflexion, euh au niveau de du brain et puis etc. euh Là, je suis en train de de revoir un petit peu dans ma tête, le système d'OCR. Et tu sais que on avait dit euh, je sais pas si tu te rappelles mais il y avait une sorte de modèle qui interrogeait le RAG, tu avais de l'ingestion euh qui faisait la découpe de shrunk. uh du small myotique et tout ça

Oui. Oui.

Et du coup euh la-la question c'était euh ok c'est super mais euh est-ce que j'ai bien pensé à mettre à créer une base de données par projet pour que en gros euh ça soit vraiment une une tour de contrôle qui permette de dire bah écoute je vais ingérer ça je vais le tester. Donc du coup je suis isolé parce que je suis pas en train de polluer toutes les bases de données ou toutes les collections. Du coup euh à chaque fois que je décide je crée une base de données je fais un projet d'injection, j'interroge l'injection si elle est correcte

Hmm. Je vois ce que tu veux dire, et oui, très bonne direction. En fait, ce que je ferais à ta place, c'est carrément isoler un dossier de travail par projet. Un workspace autonome par projet, avec sa config, ses

documents, ses chunks et ses embeddings, ses logs, etc. Et comme ça, ta base de données ne sert qu'à ce projet. Tu peux versionner, comparer plusieurs approches, et faire tes tests en parallèle. Et si tu veux ensuite fusionner ce qui a marché, tu peux toujours pousser les projets validés vers une base commune de connaissances que tu sais qui est fiable. Franchement, c'est une bonne approche, structurée et propre. Je te donnerai 5 sur 5 pour la cohérence et la clarté, surtout si ton but est de faire des tests sans contaminer le reste.

tu donnes des points comme ça, tu me fais plaisir

Tant mieux!

ok. Donc du coup, e, on avait dit quoi? que il fallait que j'ai une sorte de PRD uh et de documentation pour dire pour te rappeler ce soir uh ça va juste un mot rappeler. C'était quoi déjà

Je pense

Complètement d'accord. Le secret, ce n'est pas d'empiler des briques, c'est de les découpler. Chaque couche ne connaît que son contrat d'entrée et de sortie. Pas ses voisins. Tu peux même définir une règle simple : tout est un plugin. Activable ou non. Et piloté par config. Document, schéma, template, policy checker, brain, paradigms, méthodes, consensus, validation, knowledge state manager. Chacun renvoie un objet standardisé. Et si tu ajoutes un principe d'« architecture en couches facultatives », tu peux en enlever une sans faire s'écrouler le reste. Et je pense que les deux docs les plus importants de ton projet ne seront pas le PRD ni le pseudo-code, mais le manifeste d'architecture et le registre. Si ces deux là sont solides, tu pourras faire évoluer tout le reste sans tout casser.

Ok vas y

Minute, je lis. Parfait. On peut formaliser ça proprement. Voici un manifeste d'architecture V0. Règle 1 : tout est module, rien en dur. Règle 2, tout communique par contrat avec des entrées-sorties strictes. Règle 3, un projet égale un workspace isolé. Règle 4, le Registry sait tout, le Brain choisit, mais ne contient rien. Règle 5, Policy Checker valide toute action critique. Règle 6, le State Manager limite les recalculs. Et la règle7 : l'optimiseur de fond consolide sans perturber. Et franchement... Si tu bâtis autour de ces principes, tu n'auras pas un château de cartes, tu auras une charpente solide,

Tu veux transformer cette discussion spontanée en une **doctrine d'architecture** pour ton système OCR–RAG : une plateforme capable d'ingérer des documents, d'évaluer ce qu'elle produit, de surveiller son propre raisonnement et de ne promouvoir une connaissance qu'après validation.

Évaluation de la demande : 5/5.

Le concept est cohérent, différenciant et potentiellement valorisable. Le point critique sera de construire d'abord le socle minimal, sans activer simultanément toutes les couches cognitives.

Edit

Manifeste d'architecture cognitive

Système d'ingestion documentaire, de validation et de métacognition

1. Vision générale

Le système ne doit pas être considéré comme un simple outil d'OCR, un moteur de recherche vectorielle ou une interface permettant de poser des questions à des documents.

Il doit être conçu comme une **tour de contrôle cognitive de la connaissance**.

Sa mission n'est pas seulement de lire un document, mais de répondre à cinq questions fondamentales :

1. Qu'avons-nous réellement reçu ?
2. Qu'avons-nous correctement compris ?
3. Qu'avons-nous transformé ou interprété ?
4. Quel niveau de confiance pouvons-nous accorder au résultat ?
5. Cette connaissance peut-elle être utilisée, partagée ou intégrée à une base commune ?

Cette différence est essentielle.

Un pipeline classique produit des données.

Le système proposé produit des données, mais aussi :

- leur provenance ;
- leur état de transformation ;
- leur niveau de confiance ;
- leurs contradictions ;
- leurs limites ;
- les preuves qui ont conduit à leur validation ;
- l'historique des décisions prises sur elles.

Le système ne cherche donc pas uniquement à « savoir ».

Il cherche à **savoir ce qu'il sait, pourquoi il pense le savoir et dans quelles limites il peut l'utiliser.**

C'est cette capacité qui introduit la métacognition.

2. Principe fondateur : un projet égale un espace cognitif isolé

Chaque ingestion documentaire doit commencer par la création d'un projet autonome.

Un projet ne doit pas être un simple dossier contenant quelques fichiers. Il doit représenter un **espace cognitif isolé**, possédant ses propres règles, ressources et états.

Chaque projet contient notamment :

- ses documents sources ;
- ses paramètres d'OCR ;
- ses modèles d'extraction ;
- ses stratégies de découpage ;
- ses chunks ;
- ses métadonnées ;
- ses embeddings ;
- ses questions-réponses générées ;
- ses résultats d'évaluation ;
- ses journaux d'exécution ;
- ses coûts ;
- ses erreurs ;
- ses versions ;
- ses décisions de validation ;
- ses règles de sécurité ;
- son état de connaissance.

Cette isolation répond à un principe simple :

Une expérimentation ne doit jamais contaminer une connaissance déjà validée.

Lorsque l'on teste une nouvelle méthode de découpage, un nouveau modèle d'embedding, une nouvelle stratégie de résumé ou un nouveau prompt, les résultats doivent rester confinés dans le projet.

Ils ne doivent pas être injectés automatiquement dans une base globale.

Le projet devient ainsi un laboratoire sécurisé.

On peut :

- comparer plusieurs méthodes ;
- recommencer une ingestion ;
- supprimer une mauvaise expérience ;
- conserver différentes versions ;
- mesurer les performances ;
- auditer les décisions ;
- valider progressivement les résultats.

Une connaissance locale ne devient globale qu'après avoir franchi une procédure explicite de validation.

3. Les trois niveaux de connaissance

Le système doit distinguer trois niveaux.

3.1 Connaissance brute

Il s'agit de ce qui provient directement de la source :

- texte OCR ;
- tableaux ;
- images ;
- titres ;
- paragraphes ;
- numéros de page ;
- sections ;
- références ;
- annexes ;
- notes ;
- métadonnées du fichier.

Cette connaissance ne doit pas être modifiée silencieusement.

Elle constitue la preuve primaire.

3.2 Connaissance transformée

Il s'agit des résultats produits à partir de la source :

- texte nettoyé ;
- texte restructuré ;
- chunks ;
- résumés ;
- concepts extraits ;
- entités ;
- relations ;
- questions-réponses ;
- pseudo-code ;
- traductions ;
- classifications ;

- analyses logiques ;
- analyses philosophiques ;
- analyses maïeutiques.

Cette connaissance est utile, mais elle est interprétée.

Elle doit toujours conserver un lien vers la source dont elle provient.

3.3 Connaissance validée

Une connaissance transformée devient validée lorsqu'elle a passé les contrôles définis pour le projet :

- correspondance avec les sources ;
- présence de citations ;
- cohérence sémantique ;
- absence de contradiction critique ;
- niveau de confiance suffisant ;
- validation automatique ou humaine ;
- conformité aux politiques du projet.

Cette connaissance peut ensuite être promue vers une base de référence ou une base globale.

4. Le workspace comme capsule cognitive

Chaque projet doit disposer d'un workspace structuré.

Exemple conceptuel :

```
projects/  
└─ projet_code_du_travail/  
    │ project.manifest.json  
    │ project.config.json  
    │ policies/  
    │ sources/  
    │ ocr/  
    │ normalized/  
    │ chunks/  
    │ embeddings/  
    │ knowledge/  
    │ questions_answers/
```

```
├─ evaluations/
├─ experiments/
├─ logs/
├─ reports/
└─ state/
```

Le workspace représente la mémoire opérationnelle du projet.

Il contient tout ce qui permet de répondre aux questions suivantes :

- Quels documents ont été ingérés ?
- Avec quelle configuration ?
- À quelle date ?
- Avec quelle version du pipeline ?
- Quel modèle a réalisé chaque opération ?
- Quels résultats ont été obtenus ?
- Quels résultats ont été rejetés ?
- Pourquoi ont-ils été rejetés ?
- Qu'est-ce qui a été validé ?
- Qu'est-ce qui doit être recalculé ?

Le workspace n'est donc pas seulement un conteneur de fichiers.

Il est la **trace complète de la vie cognitive du projet**.

5. Le manifeste d'architecture

Chaque projet doit posséder un manifeste.

Le manifeste décrit l'identité et les règles fondamentales du projet.

Il peut notamment contenir :

```
{
  "project_id": "code_travail_v1",
  "name": "Base documentaire Code du travail",
  "purpose": "Construire une base juridique traçable",
  "domain": "legal",
  "criticality": "high",
  "source_policy": "official_sources_only",
  "workspace_isolated": true,
  "human_validation_required": true,
  "allowed_modules": [
```

```
"ocr",  
"normalization",  
"semantic_chunking",  
"embedding",  
"question_generation",  
"fact_validation",  
"citation_verification"  
],  
"forbidden_modules": [  
    "creative_completion",  
    "unsupported_inference"  
],  
"promotion_threshold": 0.95  
}
```

Le manifeste ne contient pas l'intelligence du système.

Il contient son cadre.

Il permet de déclarer :

- ce qui est autorisé ;
- ce qui est interdit ;
- les modules disponibles ;
- les niveaux de preuve attendus ;
- les seuils de confiance ;
- les conditions de validation ;
- les modèles utilisables ;
- les limites budgétaires ;
- les exigences de traçabilité.

Le manifeste protège le projet contre les comportements implicites.

6. Tout est module, rien n'est figé

L'architecture doit éviter les dépendances rigides.

Chaque capacité doit être implémentée sous forme de module indépendant :

- module d'importation ;
- module OCR ;
- module de nettoyage ;

- module de détection de structure ;
- module de chunking ;
- module d'embedding ;
- module de résumé ;
- module d'extraction ;
- module de génération de questions ;
- module de validation ;
- module de recherche ;
- module de synthèse ;
- module de comparaison ;
- module de consensus ;
- module de détection des contradictions ;
- module de promotion des connaissances.

Chaque module doit pouvoir être :

- activé ;
- désactivé ;
- remplacé ;
- comparé ;
- versionné ;
- testé indépendamment.

Le système ne doit pas dépendre d'un modèle précis, d'un fournisseur précis ou d'une méthode unique.

Un module doit être remplaçable tant qu'il respecte son contrat d'entrée et de sortie.

7. Les contrats cognitifs

Chaque module communique avec les autres à travers un format standardisé.

Exemple :

```
{  
  "operation_id": "op_000145",  
  "project_id": "code_travail_v1",  
  "module": "semantic_chunker",  
  "module_version": "1.2.0",  
  "input_refs": [  

```

```

    "normalized/document_001/page_024"
  ],
  "output_refs": [
    "chunks/chunk_000981"
  ],
  "status": "completed",
  "confidence": 0.93,
  "warnings": [],
  "evidence": [
    {
      "source_id": "document_001",
      "page": 24,
      "start_char": 841,
      "end_char": 2150
    }
  ],
  "metrics": {
    "duration_ms": 642,
    "input_tokens": 0,
    "output_tokens": 0,
    "estimated_cost": 0
  }
}

```

Le contrat doit permettre de connaître :

- l'origine de l'information ;
- le module responsable ;
- sa version ;
- les paramètres utilisés ;
- la sortie produite ;
- les preuves associées ;
- les avertissements ;
- le niveau de confiance ;
- les ressources consommées.

Cette standardisation rend le système composable et auditable.

8. Le Registry : celui qui sait ce qui existe

Le Registry est le registre central des capacités disponibles.

Il connaît :

- les modules installés ;
- leurs versions ;
- leurs contrats ;
- leurs dépendances ;
- leurs coûts ;
- leurs performances ;
- leurs contraintes ;
- leur état de disponibilité ;
- les types de tâches qu'ils savent traiter.

Le Registry ne choisit pas nécessairement ce qu'il faut faire.

Il répond à la question :

Quelles capacités sont actuellement disponibles pour accomplir cette tâche ?

Exemple :

```
{
  "module_id": "chunker_semantic_v2",
  "capabilities": [
    "semantic_chunking",
    "section_preservation",
    "overlap_management"
  ],
  "accepted_inputs": [
    "normalized_text"
  ],
  "produced_outputs": [
    "document_chunk"
  ],
  "estimated_cost": "low",
  "latency": "medium",
  "reliability_score": 0.94
}
```

Le Registry est la carte des outils.

Il évite que le Brain invente des capacités qui n'existent pas.

9. Le Brain : celui qui choisit sans tout contenir

Le Brain ne doit pas devenir un monolithe omniscient.

Il ne doit pas contenir tous les documents, tous les prompts, toutes les règles et tous les outils.

Son rôle est d'orchestrer.

Il analyse :

- l'intention ;
- le type de document ;
- la criticité ;
- la qualité des sources ;
- les objectifs du projet ;
- les contraintes ;
- l'état actuel du pipeline ;
- les capacités disponibles dans le Registry.

Puis il construit un plan d'exécution.

Exemple :

```
{
  "goal": "Produire une base juridique fiable",
  "plan": [
    {
      "step": 1,
      "module": "layout_extractor",
      "reason": "Le document contient une structure juridique"
    },
    {
      "step": 2,
      "module": "legal_chunker",
      "reason": "Les articles ne doivent pas être séparés ar"
    },
    {
      "step": 3,
      "module": "citation_validator",
      "reason": "Le domaine exige une traçabilité complète"
    }
  ]
}
```

```
]
}
```

Le Brain décide du chemin.

Il ne doit pas réaliser silencieusement toutes les opérations lui-même.

Cette séparation réduit les hallucinations, simplifie les tests et facilite le remplacement d'un composant.

10. Le Policy Checker : celui qui autorise ou bloque

Le Policy Checker vérifie chaque action sensible.

Il peut bloquer une opération lorsque :

- une source n'est pas autorisée ;
- une étape obligatoire a été ignorée ;
- un résultat n'est pas suffisamment sourcé ;
- le niveau de confiance est trop faible ;
- le budget est dépassé ;
- le modèle demandé n'est pas autorisé ;
- une donnée confidentielle risque de sortir du workspace ;
- une connaissance non validée tente d'être promue ;
- une réponse contient une inférence présentée comme un fait.

Exemple de décision :

```
{
  "action": "promote_to_global_knowledge",
  "decision": "blocked",
  "reasons": [
    "confidence_below_threshold",
    "missing_source_alignment",
    "human_validation_required"
  ]
}
```

Le Policy Checker est la couche de gouvernance.

Le Brain peut proposer.

Le Policy Checker décide si l'action respecte les règles.

11. Le State Manager : celui qui se souvient de l'état réel

Le State Manager suit l'avancement du projet.

Il permet de savoir :

- ce qui est terminé ;
- ce qui est en attente ;
- ce qui a échoué ;
- ce qui est obsolète ;
- ce qui doit être recalculé ;
- ce qui reste valide malgré une modification.

Exemple :

```
{
  "document_id": "document_001",
  "states": {
    "uploaded": "valid",
    "ocr": "valid",
    "normalization": "valid",
    "chunking": "stale",
    "embedding": "stale",
    "question_generation": "blocked"
  },
  "reason": "chunking_configuration_changed"
}
```

Si la stratégie de chunking change, il n'est pas nécessaire de refaire l'OCR.

Le State Manager marque uniquement les étapes dépendantes comme obsolètes.

Il permet d'éviter les recalculs inutiles et de réduire les coûts.

12. L'Optimizer de fond

L'Optimizer travaille sans modifier brutalement la connaissance active.

Son rôle est d'identifier :

- les doublons ;
- les chunks trop longs ;
- les chunks trop faibles ;
- les questions répétitives ;
- les contradictions ;
- les embeddings peu discriminants ;
- les coûts excessifs ;
- les étapes lentes ;
- les modèles sous-performants ;
- les connaissances devenues obsolètes.

Il peut proposer des améliorations, mais ne doit pas les appliquer sans contrôle dans les projets critiques.

Il produit des recommandations :

```
{
  "recommendation": "replace_chunking_strategy",
  "current_strategy": "fixed_1000_tokens",
  "suggested_strategy": "semantic_section_aware",
  "expected_benefit": {
    "retrieval_precision": "+12%",
    "answer_grounding": "+9%"
  },
  "risk": "medium",
  "automatic_application": false
}
```

L'Optimizer améliore le système sans perturber son fonctionnement courant.

13. Le cycle complet d'ingestion

Étape 1 — Création du projet

Le système crée :

- un identifiant ;

- un workspace ;
- un manifeste ;
- une configuration ;
- un journal ;
- un état initial.

Étape 2 — Réception des sources

Le système accepte différents types d'entrée :

- PDF ;
- image ;
- document bureautique ;
- texte collé ;
- page HTML ;
- archive ;
- flux API.

Chaque source reçoit :

- un identifiant ;
- une empreinte ;
- une date ;
- une provenance ;
- une licence éventuelle ;
- un niveau de confiance ;
- un statut.

Étape 3 — Préservation de la source originale

La source brute est stockée sans modification.

Une empreinte cryptographique permet de prouver qu'elle n'a pas changé.

Étape 4 — Extraction

Le pipeline extrait :

- le texte ;
- la structure ;
- les tableaux ;
- les images ;
- les titres ;

- les notes ;
- les références ;
- les numéros de page.

Étape 5 — Normalisation

Le texte est nettoyé sans effacer la provenance :

- correction d'erreurs OCR ;
- suppression d'artefacts ;
- reconstruction des paragraphes ;
- conservation des sections ;
- harmonisation des caractères ;
- détection de langue.

Chaque correction importante doit être traçable.

Étape 6 — Découpage

Le document est découpé selon sa nature :

- découpage structurel ;
- découpage sémantique ;
- découpage par article ;
- découpage par chapitre ;
- découpage par question ;
- découpage hybride.

Le système ne doit pas appliquer aveuglément une taille fixe à tous les documents.

Une loi, une notice technique et un roman ne possèdent pas la même structure cognitive.

Étape 7 — Enrichissement

Chaque chunk peut recevoir :

- un titre ;
- un résumé ;
- des mots-clés ;
- des entités ;
- des concepts ;
- des relations ;

- un niveau de difficulté ;
- un contexte antérieur ;
- un contexte suivant ;
- une fonction dans le document ;
- des questions auxquelles il peut répondre.

Étape 8 — Embedding et indexation

Le chunk enrichi est transformé en représentation vectorielle.

Mais l'embedding n'est jamais considéré comme la source.

Il n'est qu'un index permettant de retrouver la source.

Étape 9 — Génération de connaissances secondaires

Le système peut générer :

- questions-réponses ;
- fiches ;
- résumés ;
- tableaux ;
- graphes ;
- pseudo-code ;
- explications ;
- scénarios ;
- contre-exemples.

Ces connaissances secondaires doivent être liées aux passages qui les justifient.

Étape 10 — Validation

Chaque production est comparée aux sources.

Le système vérifie :

- si la réponse est réellement supportée ;
- si une citation est correcte ;
- si une information a été inventée ;
- si une nuance importante a disparu ;
- si plusieurs sources se contredisent ;
- si une réponse mélange faits et interprétations.

Étape 11 — Test par interrogation

Le projet est interrogé avec une batterie de questions :

- questions directes ;
- questions ambiguës ;
- questions profondes ;
- questions contradictoires ;
- questions hors périmètre ;
- questions impossibles ;
- questions nécessitant plusieurs sources ;
- questions contenant une fausse prémisse.

Le système doit être capable de répondre :

Je ne dispose pas d'une preuve suffisante dans les documents du projet.

Cette réponse vaut mieux qu'une réponse plausible mais inventée.

Étape 12 — Promotion

Une fois validée, la connaissance peut être promue vers :

- une collection certifiée ;
- une base commune ;
- un environnement de production ;
- un catalogue commercial ;
- une API client.

La promotion doit être explicite, enregistrée et réversible.

14. La métacognition appliquée

La métacognition ne signifie pas que la machine possède une conscience.

Elle signifie que le système dispose de mécanismes permettant de surveiller ses propres opérations.

Il doit être capable de déclarer :

- ce qu'il est en train de faire ;

- pourquoi cette méthode a été choisie ;
- quelles informations sont utilisées ;
- quelles informations manquent ;
- ce qui relève d'un fait ;
- ce qui relève d'une interprétation ;
- ce qui relève d'une hypothèse ;
- quel est son niveau de confiance ;
- quelle vérification supplémentaire serait nécessaire.

Exemple :

```
{
  "answer": "Le document impose deux jours de repos consécutif",
  "knowledge_status": "supported",
  "confidence": 0.97,
  "supporting_sources": [
    {
      "document_id": "document_003",
      "page": 8,
      "chunk_id": "chunk_0045"
    }
  ],
  "reasoning_status": {
    "missing_information": [],
    "contradictions": [],
    "inferences": [],
    "external_knowledge_used": false
  }
}
```

La métacognition devient ainsi une fonction d'audit.

15. Les états épistémiques

Chaque information peut recevoir un état épistémique.

Exemples :

- `observed` : directement présent dans la source ;
- `extracted` : identifié automatiquement ;
- `interpreted` : déduit par analyse ;
- `inferred` : conclusion indirecte ;

- `hypothetical` : hypothèse de travail ;
- `contradicted` : contradiction détectée ;
- `uncertain` : preuves insuffisantes ;
- `validated` : vérification réussie ;
- `rejected` : information non retenue ;
- `obsolete` : information dépassée.

Cette taxonomie empêche de mélanger des informations de natures différentes.

Un fait observé ne doit pas avoir le même statut qu'une interprétation philosophique.

16. Les questions métacognitives du Brain

Avant d'agir, le Brain doit pouvoir appliquer une grille de contrôle.

Compréhension

- Quel est l'objectif réel ?
- Quel livrable est attendu ?
- Quel domaine est concerné ?
- Quelle est la criticité de l'erreur ?

Connaissance

- Quelles informations possédons-nous ?
- Quelle est leur provenance ?
- Quelles informations manquent ?
- Devons-nous interroger le RAG, le Web ou un humain ?

Méthode

- Quelle stratégie est adaptée ?
- Quels modules sont nécessaires ?
- Quels modules seraient inutiles ?
- Faut-il un traitement déterministe ou génératif ?

Risque

- Une hallucination serait-elle grave ?

- La réponse exige-t-elle une citation ?
- Une validation humaine est-elle obligatoire ?
- Des données sensibles sont-elles présentes ?

Évaluation

- Le résultat répond-il réellement à la demande ?
- Les preuves sont-elles suffisantes ?
- Une contradiction subsiste-t-elle ?
- Le résultat peut-il être promu ?

Cette grille ne doit pas nécessairement être exécutée par un seul grand modèle.

Elle peut être traduite en règles, en contrôles déterministes et en appels ciblés.

17. Maïeutique documentaire

La maïeutique ne doit pas uniquement servir à questionner l'utilisateur.

Elle peut servir à interroger le document lui-même.

Pour chaque section, le système peut demander :

- Que prétend exactement ce passage ?
- À quelle question répond-il ?
- Sur quelles prémisses repose-t-il ?
- Quelles notions suppose-t-il déjà connues ?
- Quelles conséquences entraîne-t-il ?
- Quels cas particuliers sont exclus ?
- Quelle ambiguïté pourrait provoquer une mauvaise interprétation ?
- Quelle question importante reste sans réponse ?

Le document ne devient plus un bloc passif.

Il devient un objet que le système examine méthodiquement.

18. Analyse critique et « gant de Kant »

Une couche critique peut tester chaque connaissance à travers plusieurs angles :

Validité

La conclusion découle-t-elle réellement des prémisses ?

Universalité

La règle reste-t-elle valable si elle est appliquée à tous les cas similaires ?

Limites

Dans quelles circonstances l'affirmation cesse-t-elle d'être vraie ?

Contradiction

Une autre partie du document affirme-t-elle l'inverse ?

Finalité

Quel objectif ou quelle intention semble guider cette règle ?

Responsabilité

Quel serait l'impact d'une interprétation ou d'une utilisation erronée ?

Cette couche ne doit pas transformer automatiquement les interprétations en faits.

Elle doit produire une analyse distincte, marquée comme telle.

19. Consensus, dissensus et arbitrage

Lorsque plusieurs modèles ou plusieurs sources sont utilisés, le système ne doit pas rechercher artificiellement un consensus.

Il doit produire trois ensembles :

Points d'accord

Éléments soutenus par plusieurs sources ou analyses.

Points de désaccord

Éléments pour lesquels les sources ou modèles divergent.

Points indéterminés

Éléments pour lesquels aucune preuve suffisante n'existe.

Le désaccord ne représente pas nécessairement une erreur.

Il peut révéler :

- une ambiguïté ;
- une évolution dans le temps ;
- une différence de définition ;
- une exception ;
- une opposition doctrinale ;
- une mauvaise qualité documentaire.

L'arbitre final doit s'appuyer sur la hiérarchie des preuves, et non sur le simple nombre de modèles en accord.

20. La promotion vers une base globale

La base globale ne doit pas être une poubelle dans laquelle tous les résultats sont versés.

Elle doit contenir uniquement des connaissances qualifiées.

Une promotion peut exiger :

- une source identifiable ;
- une provenance intacte ;
- une validation réussie ;
- un niveau de confiance minimal ;
- aucune contradiction critique non résolue ;
- une date de validité ;
- une catégorie ;
- une politique d'expiration ;
- une décision humaine pour les domaines sensibles.

Le système peut utiliser plusieurs niveaux :

LABORATOIRE



CONNAISSANCE CANDIDATE



CONNAISSANCE VALIDÉE



CONNAISSANCE CERTIFIÉE



BASE GLOBALE

Chaque passage d'un niveau au suivant doit être enregistré.

21. Anti-contamination cognitive

Le système doit protéger la base contre plusieurs formes de contamination :

- hallucinations ;
- erreurs OCR ;
- doublons ;
- sources obsolètes ;
- interprétations non signalées ;
- connaissances provenant du Web sans validation ;
- réponses générées réinjectées comme si elles étaient des sources ;
- mélange entre plusieurs clients ;
- mélange entre plusieurs versions d'un document ;
- changement d'embedding non maîtrisé ;
- résumés ayant perdu une exception importante.

Règle fondamentale :

Une sortie générée ne devient jamais automatiquement une nouvelle source primaire.

Elle reste une connaissance dérivée.

Cette distinction protège la chaîne de preuve.

22. Architecture en couches facultatives

Le système doit fonctionner même si certaines couches sont désactivées.

Exemple minimal :

```
Document
→ Extraction
→ Normalisation
→ Chunking
→ Indexation
→ Recherche
```

Exemple enrichi :

```
Document
→ Extraction
→ Normalisation
→ Analyse de structure
→ Chunking sémantique
→ Enrichissement
→ Embedding
→ Génération de questions-réponses
→ Validation croisée
→ Analyse maïeutique
→ Analyse critique
→ Test d'interrogation
→ Promotion
```

Le pipeline minimal permet de livrer rapidement une première version.

Les couches cognitives peuvent ensuite être ajoutées progressivement.

Cette approche évite le château de cartes.

23. Principe d'anti-fragilité

Le système ne doit pas seulement résister aux erreurs.

Il doit apprendre de leur apparition.

Lorsqu'une erreur est détectée, elle doit pouvoir produire :

- un cas de test ;
- une règle de détection ;
- une amélioration de politique ;
- une mise à jour du registre ;
- un signal de baisse de confiance ;
- une recommandation de configuration.

Exemple :

```
{  
  "incident": "article_split_between_two_chunks",  
  "root_cause": "fixed_size_chunking",  
  "corrective_action": "use_legal_structure_chunker",  
  "new_regression_test": "legal_article_integrity_test"  
}
```

L'erreur devient ainsi une donnée d'amélioration.

24. Traçabilité et audit

Pour toute réponse, le système doit pouvoir reconstruire :

1. la question reçue ;
2. l'intention détectée ;
3. les politiques appliquées ;
4. les sources interrogées ;
5. les chunks récupérés ;
6. les scores de recherche ;
7. les modèles appelés ;
8. les transformations effectuées ;
9. les contradictions détectées ;
10. la réponse finale ;
11. le niveau de confiance ;
12. les coûts et la durée.

L'objectif n'est pas d'exposer une chaîne de pensée privée.

L'objectif est de fournir une **chaîne de décision vérifiable**.

Cette distinction est importante.

Une chaîne de décision décrit les opérations, les preuves et les règles.

Elle ne prétend pas reproduire les pensées internes d'un modèle.

25. Exemple de scénario complet

Un utilisateur dépose un PDF de 200 pages.

Le système crée un projet isolé.

Il identifie le document comme une documentation technique.

Le Brain consulte le Registry et sélectionne :

- extraction de mise en page ;
- OCR ;
- détection de tableaux ;
- chunking par chapitre et procédure ;
- extraction des avertissements ;
- génération de questions-réponses ;
- validation par citation.

Le Policy Checker impose :

- conservation du fichier original ;
- interdiction de compléter les procédures avec des connaissances externes ;
- citation obligatoire des pages ;
- validation renforcée des consignes de sécurité.

Le pipeline produit :

- 420 chunks ;
- 170 concepts ;
- 1 200 questions-réponses ;
- 38 procédures structurées ;
- 74 avertissements ;
- 16 contradictions ou ambiguïtés potentielles.

Le système teste ensuite la base.

Une question demande une procédure absente du manuel.

La réponse correcte est :

Cette procédure n'est pas décrite dans les documents actuellement validés. Une source complémentaire est nécessaire.

Cette abstention constitue une réussite du système.

26. Métriques essentielles

Le système doit mesurer autre chose que le nombre de documents ingérés.

Qualité documentaire

- taux d'erreurs OCR ;
- taux de structure préservée ;
- taux de tableaux correctement extraits ;
- taux de chunks liés à une page précise.

Qualité de recherche

- précision du retrieval ;
- rappel ;
- nombre de chunks inutiles ;
- qualité du classement ;
- taux de bonnes sources dans le top K.

Qualité des réponses

- taux de réponses supportées ;
- taux de citations exactes ;
- taux d'abstention correcte ;
- taux d'hallucination ;
- taux de contradictions détectées.

Performance

- temps d'ingestion ;
- coût par document ;
- coût par page ;
- coût par question ;

- consommation mémoire ;
- latence de recherche.

Gouvernance

- pourcentage de connaissances validées ;
 - pourcentage de connaissances rejetées ;
 - nombre de promotions ;
 - nombre de retours arrière ;
 - nombre d'interventions humaines.
-

27. Les règles cardinales

Règle 1 — Tout est module

Aucune capacité essentielle ne doit être enfermée dans un composant impossible à remplacer.

Règle 2 — Tout communique par contrat

Chaque entrée, sortie, erreur et preuve possède une structure définie.

Règle 3 — Un projet égale un workspace isolé

Aucune expérimentation ne contamine une base validée.

Règle 4 — Le Registry sait ce qui existe

Il décrit les capacités disponibles et leurs contraintes.

Règle 5 — Le Brain choisit

Il construit le plan, mais ne contient pas tout le système.

Règle 6 — Le Policy Checker autorise

Toute opération critique doit respecter les règles du projet.

Règle 7 — Le State Manager connaît l'état réel

Le système ne recalcule pas inutilement et sait ce qui est encore valide.

Règle 8 — La source reste souveraine

Toute connaissance dérivée conserve un lien avec les preuves originales.

Règle 9 — L'incertitude doit être visible

Le système doit savoir dire qu'il ne sait pas.

Règle 10 — La promotion est méritée

Une connaissance ne devient globale qu'après validation.

Règle 11 — L'audit doit être reproductible

Chaque résultat important doit pouvoir être expliqué par ses sources, ses règles et ses opérations.

Règle 12 — L'optimisation ne doit pas casser la production

Les améliorations sont testées dans un espace isolé avant déploiement.

28. Conclusion

Cette architecture transforme l'ingestion documentaire en processus cognitif gouverné.

Le système ne se contente pas de découper des documents et de calculer des embeddings.

Il organise un cycle complet :

Recevoir

→ Préserver

→ Extraire

→ Structurer

→ Comprendre

→ Enrichir

→ Interroger

→ Vérifier

→ Douter

→ Valider

- Promouvoir
- Surveiller
- Améliorer

La véritable valeur du projet ne se trouve pas uniquement dans la puissance des modèles utilisés.

Elle se trouve dans la structure qui les oblige à travailler proprement.

Les modèles peuvent changer.

Les fournisseurs peuvent changer.

Les embeddings peuvent changer.

Les méthodes peuvent évoluer.

Mais si les contrats, le manifeste, le registre, les politiques, les états et les preuves sont solides, l'ensemble reste maîtrisable.

Ce système n'est donc pas un château de cartes composé d'intelligences artificielles empilées.

C'est une charpente cognitive dans laquelle chaque composant possède une fonction, une limite et une responsabilité.

Sa promesse centrale peut être formulée ainsi :

Ne pas seulement produire une réponse, mais produire une connaissance dont on peut retracer l'origine, mesurer la fiabilité, contrôler l'utilisation et réviser la validité.

La prochaine transformation logique consiste à convertir ce manifeste en **PRD technique**, avec les modules, fichiers, schémas JSON, statuts et critères d'acceptation de la première version.

